

Central Flagging Mechanism

From fragmented project flags to a namespaced platform of record

Problem

Duplicated flags, drift, and operational risk across apps

Approach

One control plane with project-name namespaces

Outcome

Consistent evaluation, auditability, faster delivery

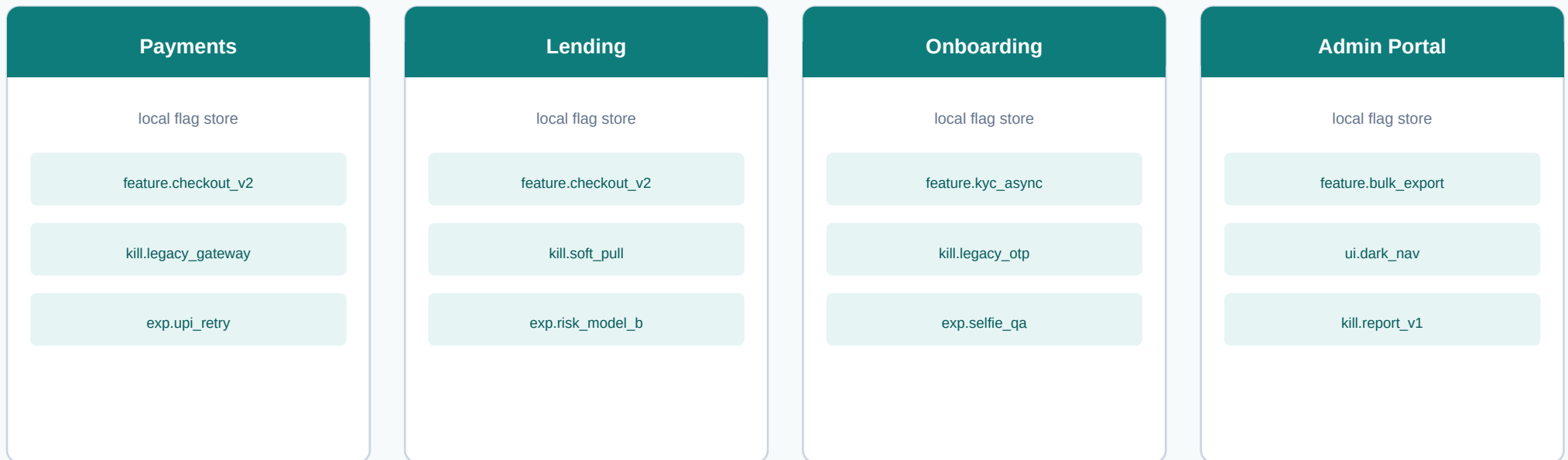
Agenda

What we will cover

- 01 Current state — individual / single-app flagging
- 02 Pros & cons of single-app flagging
- 03 Sequence: how per-project flags work today
- 04 Drawbacks when the same flags are copied across apps
- 05 .NET central flagging app (namespaced control plane)
- 06 How the central model closes every gap
- 07 Pros, operating model, and recommendation

Current State: Individual Project Flags

Each application owns its own flag definitions, storage, and evaluation



Observation

Same semantic flags (e.g. checkout_v2) reappear with different owners, lifecycles, and truth — no shared control plane.

Single-App Flagging: Pros & Cons

Honest trade-offs before you outgrow in-app toggles

Pros

Simple to start

appsettings / small table ships with v1

Low latency

In-process evaluation, no network hop

No shared dependency

App survives if a platform is down

Local ownership

Team invents keys without coordination

Fit for one deployable

Ideal when there is a single product surface

Cons

Fragmented truth

N places to ask “is this flag on?”

Slow incident kills

Waits on deploys / per-repo PRs

Inconsistent semantics

Different caches, defaults, SDKs

Weak governance

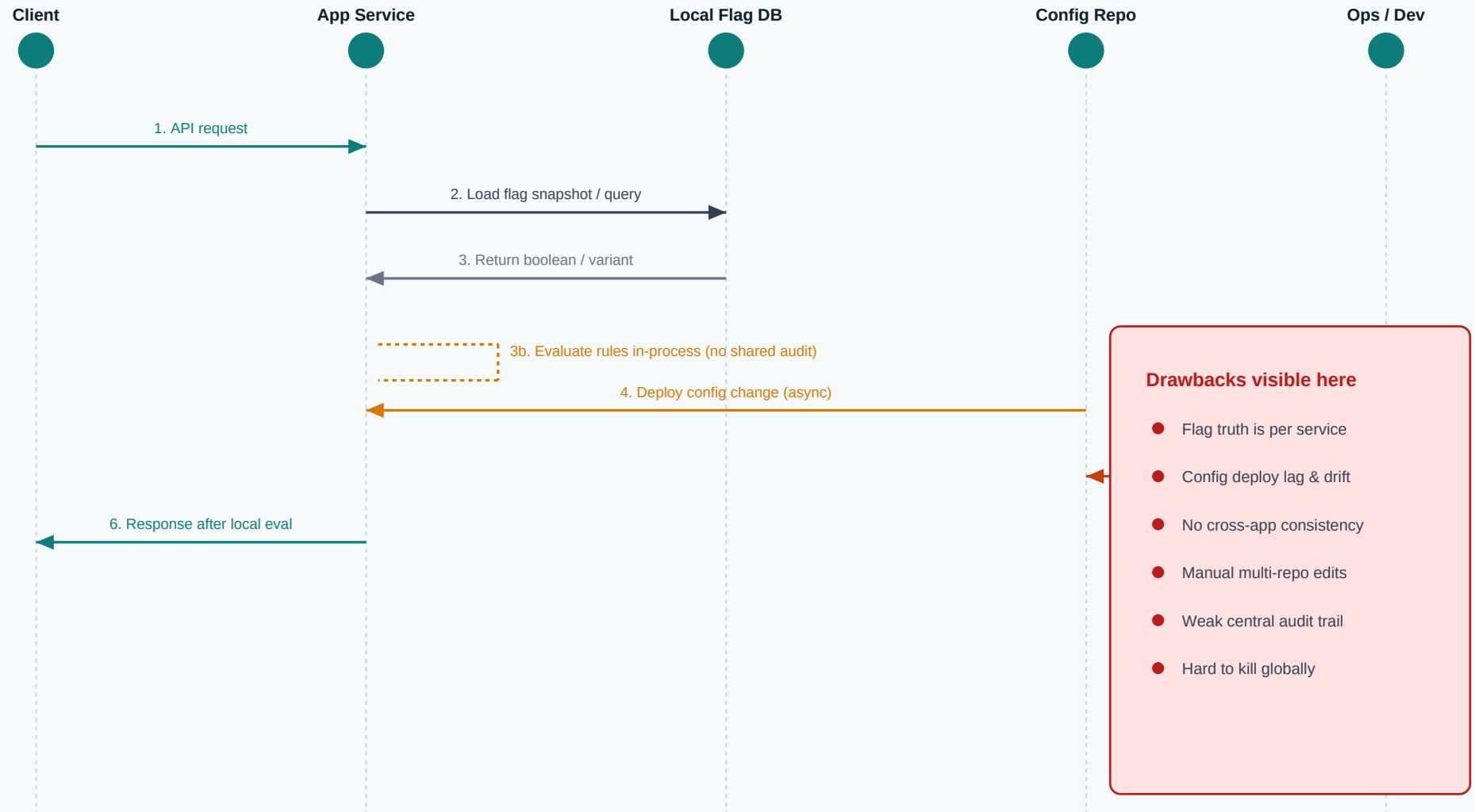
No shared RBAC / audit estate-wide

Cross-app drift

Copied keys diverge in meaning & life

Sequence: Individual Flagging Mechanism

Request path when every project evaluates flags locally



Drawbacks: Individual Project Flags

Why per-project ownership does not scale

Fragmented truth

Each app is its own source of truth. Operators cannot answer “is X on?” without checking N systems.

Operational toil

Enable/disable requires code deploys, repo PRs, or hopping between consoles — slow during incidents.

Inconsistent SDKs

Different libraries, caching TTLs, and evaluation semantics produce divergent user experiences.

Poor governance

No unified RBAC, approval workflow, or change audit across the estate.

Limited targeting

Percentage rollouts, cohorts, and environment promotion are reinvented (or missing) per project.

Cost & complexity

Duplicate storage, pipelines, and tribal knowledge increase maintenance load.

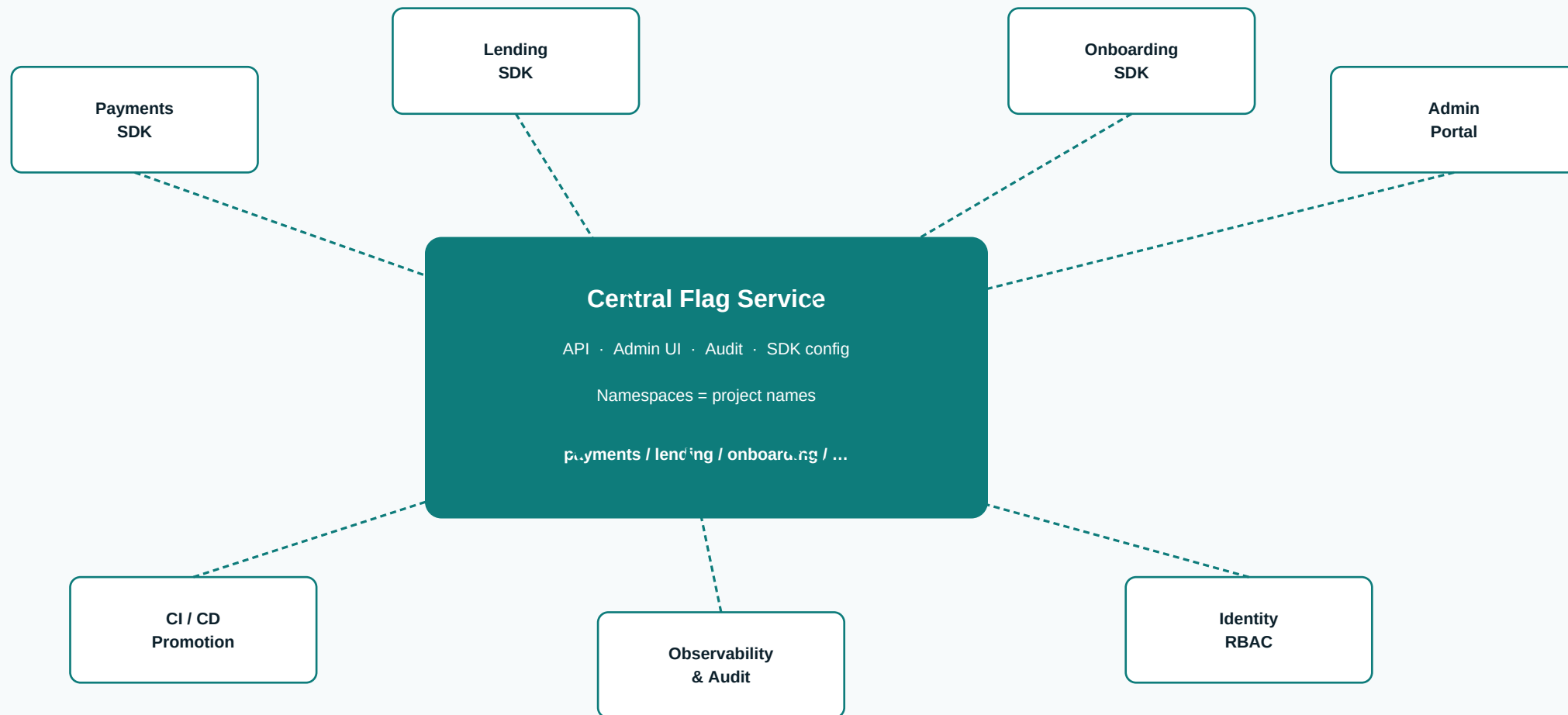
Drawbacks: Same Flags Across Different Projects

Copy-paste “shared” flags create a false sense of consistency

Concern	What happens today	Business impact
Semantic drift	Same key name, different meaning or default per repo	Wrong cohort sees feature; support tickets spike
Lifecycle mismatch	Flag retired in one app, still live in another	Dead code paths; surprise behavior in prod
Partial rollout	Turned on in Payments, forgotten in Lending	Broken journeys across product boundaries
Security gaps	Sensitive kill-switch not mirrored everywhere	Incident blast radius stays open longer
Analytics noise	Events tagged inconsistently for “same” experiment	Invalid A/B conclusions and wasted spend
Ownership fog	No single owner for a cross-cutting flag	Change freezes and slow decisioning

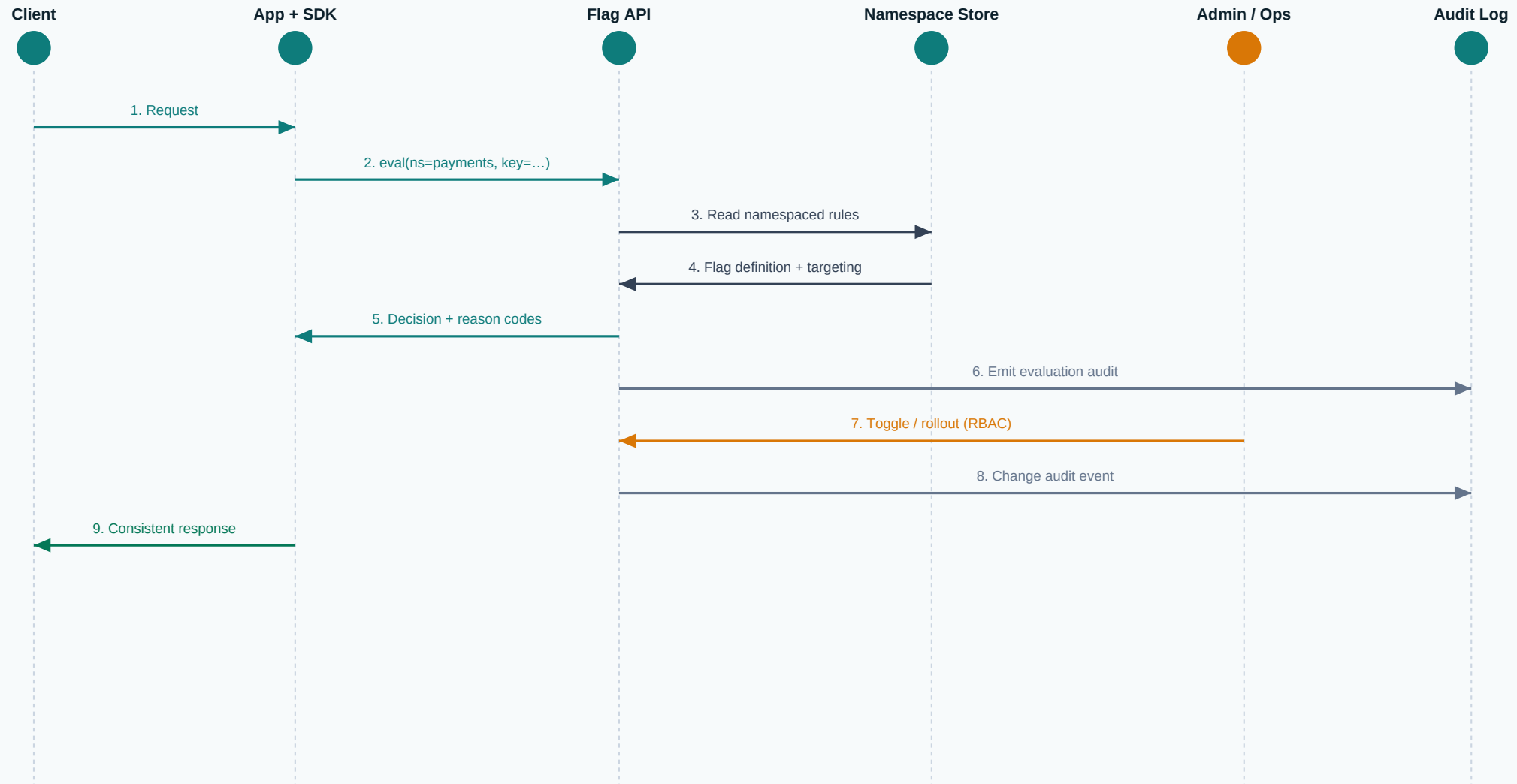
Solution: Central Flagging Application

One control plane · project-name namespaces · shared evaluation



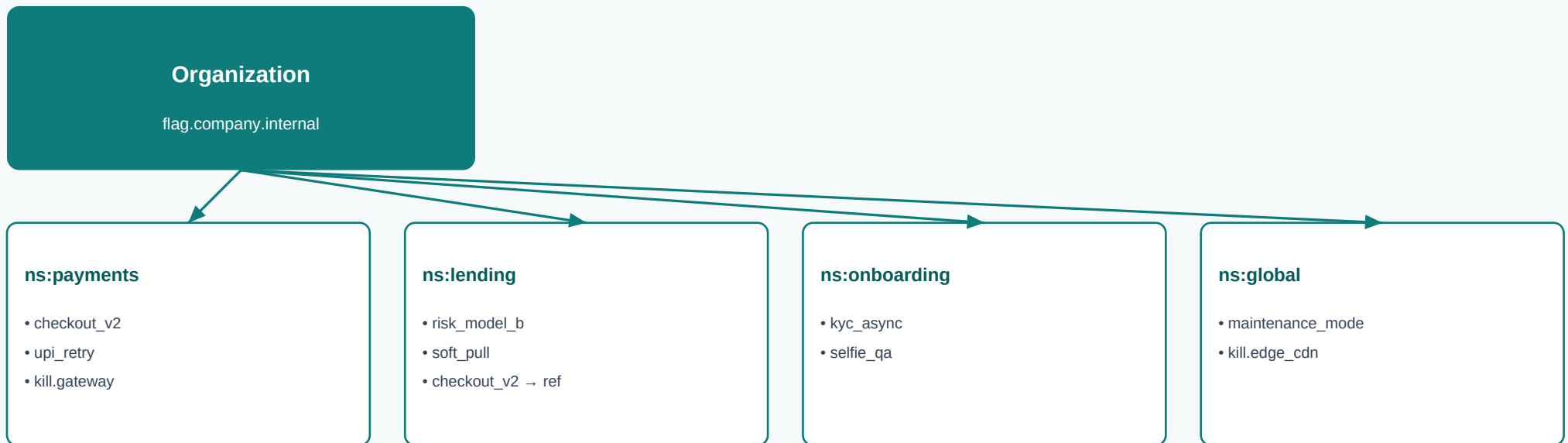
Sequence: Central Flagging (Namespaced)

Single evaluation path with project namespace isolation and shared governance



Data Model: Project Namespaces

Isolation where you need it · sharing where you intend it



- Default: flags live under the owning project namespace (hard isolation).
- Optional: explicit references or global namespace for true cross-cutting kill switches.
- SDK always calls with project namespace — no accidental cross-project reads.
- Admin UI scopes operators to namespaces via RBAC.

Target Architecture (Logical View)

Control plane, data plane, and project namespaces

EXPERIENCE

Admin UI

Change Approvals

Dashboards

CONTROL PLANE

Flag API

AuthZ / RBAC

Webhook / Events

DATA PLANE

Eval Edge / CDN

SDK Cache

Streaming Sync

PERSISTENCE

Namespace Store

Audit Log

Segments

Consumers (SDK): payments · lending · onboarding · admin · ...each bound to its namespace

Eval path is local-first (cached) with central authority for definitions — low latency, strong consistency of config.

Closing Every Drawback

Traceability from problem → central capability

Drawback (today)	Fulfilled by central flagging
Fragmented truth	→ Single platform of record with namespaced keys
Deploy toil / slow kills	→ Admin API + UI toggles without app redeploy
SDK inconsistency	→ One SDK / one evaluation contract for all apps
Weak governance	→ RBAC, approvals, environments, full audit log
Copied flag drift	→ One definition; projects subscribe by namespace/ref
Partial cross-app rollout	→ Coordinated targeting or explicit global flags
Ownership fog	→ Namespace owners + mandatory metadata
Analytics noise	→ Stable flag IDs & reason codes in telemetry

Pros of the Central Flagging Application

What teams gain once the control plane exists

Consistency

Identical evaluation semantics across every project namespace.

Speed

Instant kill switches and progressive delivery without waiting on deploys.

Safety

Environment promotion, approvals, and blast-radius controls.

Clarity

Namespace ownership makes who-owns-what unambiguous.

Observability

Central audit of changes and evaluation samples.

Reuse

Shared building blocks (targeting, segments, schedules) once.

Cost control

Retire N flag stores / consoles / bespoke scripts.

Product velocity

Experimentation becomes a platform capability, not a one-off.

RECOMMENDATION

Adopt a central flagging platform

with project-name namespaces as the isolation boundary

- Individual flag stores create fragmented truth, slow incidents, and inconsistent UX.
- Duplicating the “same” flags across repos guarantees drift, partial rollouts, and ownership fog.
- A central application restores a single control plane while namespaces preserve project autonomy.
- Every listed drawback maps to a concrete platform capability — not a process workaround.

Next step

Stand up the central service + SDK, migrate one critical namespace (e.g. payments), then roll out by project.